

2.1 Control Structure.

We have developed till this point are executed in sequential manner. But practically there is a need to change the flow of execution of the statements in the program. For achieving it, following types of control structures may be used.

- Decision making and branching statements
- Looping statements.

Relational operators and logical operators are prominently used along with these control structures.

1) Relational Operators.

The relational operators are used to determine the relationship of quantity to another. They always return 1 or 0 depending upon the outcome of the test. The relational operators are as follows:

Operator	Description	Example.
==	Checks if the values of two operands are equal or not. If yes, then the condition become true.	(A==B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then condition become true.	(A!=B) is true.

Operator	Description	Example
>	Checks if the value of left operand is greater than the value of right operand. If yes, then condition becomes true	(A > B) is not true.
<	checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true	(A < B) is true.
>=	If the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true	(A >= B) is not true
<=	If the value of left operand is less than or equal to the value of right operand. If yes, then the condition become true	(A <= B) is true.

2) Logical Operators

Logical Operators are usually used with conditional statements. The three basic logical operators are:-

- 1) && for logical AND
- 2) || for logical OR
- 3) ! for not.

Q6. The truth table for the logical operator are:-

X	Y	$X \& Y$	$X Y$	$! X$	$! Y$
0	0	0	0	1	1
0	1	0	1	1	0
1	0	0	1	0	1
1	1	1	1	0	0

Operator	Description	Example	Output
$\&$	If both the operands are true then condition become true.	$(8 > 9) \& (2 < 9)$ $(9 > 8) \& (2 < 9)$	False True
$ $	If any of the two operands are non-zero then condition becomes true.	$(2 \neq 2) (9 < 20)$ $(3 \neq 3) (9 > 20)$	True False
$!$	used to reverse the logical state of its operand	$!(8 > 2)$ $!(2 > 10)$	False True

* Conditional Statement.

1) if statement

The if statement is the simplest form of decision control statement.

It allows the program to test a condition and execute a statement only if the condition is true.

General Form (Syntax)

```

if (condition)
{
    statement;
}
  
```

If the condition is true, the statement inside the if block is executed.

If the condition is false (zero), the statement is skipped.

Control then passes to the next statement after the if block.

Flow of Execution

- Condition checked.
- True → Execute Statement
- False → Skip Statement.

(This is why if is also called a single-decision statement).

Example :-

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{  
    int a = 10;
```

```
    clrscr();
```

```
    if (a > 5)
```

```
{
```

```
    printf("A is greater than 5");
```

```
}
```

```
    getch();
```

```
}
```

Output :-

A is greater than 5.

2) if-else statement

When there are two alternative, at that time if-else statement is used

General Form (Syntax)

```
if (test-condition)
```

```
{
```

```
    state-block1;
```

```
}
```

```
else
```

```
{
```

```
    state-block2;
```

```
}
```

```
Statx;
```


When 'test-condition' is true (i.e. value is 1), then program counter goes execute 'state-block 1' and after that it directly goes to execute 'state-x' by skipping state-block 2.

When test condition is False (i.e. value is 0) then program counter goes to execute 'state-block 2' directly by skipping 'state-block 1'.

Flow of Execution.

- Condition checked
- True $\rightarrow$ if block executes
- False $\rightarrow$ else block executes.

(This makes if-else a double-decision control statement)

Example :-

Check whether given number is even or odd.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int n;
```

```
    clrscr();
```

```
    printf("\n Enter Number =");
```

```
    scanf("%d", &n);
```

```
    if (n % 2 == 0)
```

```
    {
```

```
        printf("\n Given no %d is even", n);
```

```
    } else
```

```
    {
```

```
        printf("\n Given no %d is odd", n);
```

```
    } getch();
```

```
}
```


3) Nested if-else Statement

When we write one 'if-else' statement inside the 'if' part of another if-else statement such type of situation is called nested if-else statement.

General Form (Syntax)

```
if (test-condition1)
{
```

```
    if (test-condition2)
```

```
    {
```

```
        Statement 1;
```

```
    }
```

```
    else
```

```
    {
```

```
        Statement 2;
```

```
    }
```

```
}
```

```
else
```

```
{
```

```
    Statement 3;
```

```
}
```

```
Statement-x ;
```

When an if-else statement is written inside another if or else statement, it is called a nested if-else statement.

Working:

- The outer if condition is evaluated first.
- If it is true, the inner if-else is evaluated.
- If the outer condition is false, the control goes directly to the outer else block.
- Only one block of statement is executed based on the condition.

Flow of execution :-

- Check condition 1
- True - Check Condition 2
- False - Execute outer else

• Execute only one decision path.

Example Largest of Two Numbers

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int a = 10;    b = 20;
```

```
    if (a != b)
```

```
    {
```

```
        if (a > b)
```

```
        {
```

```
            printf("a is greater");
```

```
        else
```

```
        {
```

```
            printf("B is greater");
```

```
        }
```

```
    }
```



```

else
{
    printf("Both are equal");
}
getch();
}

```

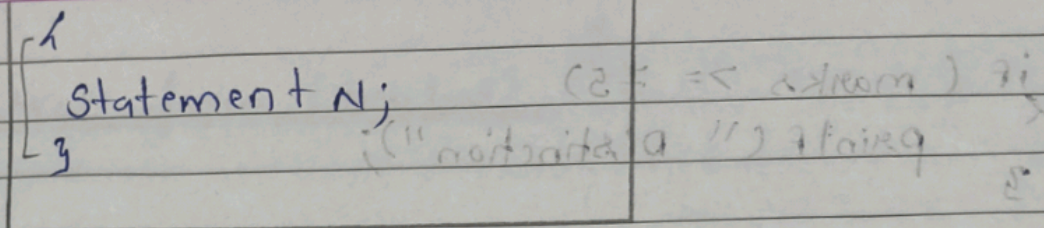
4) if - else Ladder

- When a program has to choose one option from several alternatives, the else-if ladder is used.
- It consists of sequence of if and else if statements, followed by an optional else statement.
- General Form (Syntax)

```

if (Condition 1)
{
    Statement 1;
}
else if (Condition 2)
{
    Statement 2;
}
else if (Condition 3)
{
    Statement 3;
}
else

```

- Conditions are evaluated from top to bottom
- This first condition that evaluates to true (non-zero) is executed.
- Once a true condition is found, the remaining conditions are skipped.
- If none of the conditions are true, the else block is executed.
- Only one block of statement is executed.

Flow of Execution.

Check condition 1

if false $\rightarrow$ check condition 2

Continue until condition is true

if all condition are false $\rightarrow$ execute else.

Example:-

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int marks = 82;
```

```
clrscr();
```



```

if (marks >= 75)
    printf("Distinction");
}

```

```

else if (marks >= 60)
    printf("First class");
}

```

```

else if (marks >= 50)
{
    printf("Second class");
}

```

```

else if (marks >= 40)
{
    printf("Pass");
}

```

```

else
{
    printf("Fail");
}

```

```

    getch();
}

```

5) Switch - Case - Statement

When there are so many alternative is to be selected in the program, at that time switch-case statement is suitable.

Set of if-else statement can be replaced by single switch case statement.

* General form (syntax)

```
Switch ( expression )
```

```
{
```

```
    case Value 1 : Stat-block 1;
                  break;
```

```
    case Value 2 : Stat-block 2;
                  break;
```

```
    ⋮
```

```
    case Value n : Stat-block-n;
                  break;
```

```
    default : default-block;
```

```
}
```

```
statement-x;
```

When value of 'expression' is 'Value 1', the P.C. goes to execute 'stat-block 1' & after that Program Counter (P.C.) goes to execute Stat-x.

When value of 'expression' is 'value 2' then P.C. goes to execute stat-block 2 & after that P.C. goes to execute 'Stat-x'.

When value of 'expression' is not match to Value 1, Value 2 ... then PC goes to execute default-block & then PC goes to out of switch.

statement:

Example :-

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int day = 3;
```

```
    switch (day)
```

```
    {
```

```
        case 1:
```

```
            printf("Monday");
```

```
            break;
```

```
        case 2:
```

```
            printf("Tuesday");
```

```
            break;
```

```
        case 3:
```

```
            printf("Wednesday");
```

```
            break;
```

```
        case 4:
```

```
            printf("Thursday");
```

```
            break;
```

```
        default:
```

```
            printf("Invalid");
```

```
    }
```

```
    getch();
```

```
}
```


- The value of day is compared with each case.
- When day matches case 3, "Wednesday" is printed.
- break prevents fall-through to the next cases.
- default executes if no case matches

On 2.2 Looping Statement

It is a programming technique through which a group of statements is executed repeatedly, until certain specified condition is satisfied.

Looping is also called a repetition or iterative control mechanism.

C provides three types of loop control structures.

They are :-

- For Statement
- While Statement
- do-while Statement.

i) For Statement

A loop is used to execute a block of statements repeatedly as long as a specified condition is satisfied.

General Form (Syntax)

```
For (initialization; condition; increment/
    decrement)
```

```
{
```

```
    Statement(s);
```

```
}
```

Working of For Loop

1. Initialization is executed first and only once.
2. The condition is evaluated.
 - If the condition is true (non-zero), the loop body is executed.
 - If the condition is False (zero), the loop terminates.
3. After executing the loop body, the increment/decrement statement is executed.
4. Control again goes to the condition check.
5. This process continues until the condition becomes False.

Flow of Execution

Initialization → Condition check

True → Execute loop body → increment/decrement

→ Condition check

False → Exit loop

Example 1: Printing Numbers from 1 to 5

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int i;
```

```
    for (i = 1; i <= 5; i++)
```

```
    {
```

```
        printf("%d", i);
```

```
    }
```

```
    getch();
```

```
}
```

2) While loop statement

- 1) When we want to execute set of instruction again and again until condition is true at that time while statement is used.

2) General form (syntax).

```
while (test-condition)
```

```
{
```

```
    body of while loop
```

```
}
```


3) It is an entry-controlled loop, where the condition is tested before the execution of the loop body.

4) Working of while loop.

1. The condition is evaluated first.
2. If the condition is true, the loop body is executed.
3. After executing the loop body, control goes back to the condition.
4. This process continues until the condition becomes False.

5. When the condition becomes False, the loop terminates.

5) Flow of Execution.

- Condition check
- True $\rightarrow$ Execute loop body $\rightarrow$ Condition check
- False $\rightarrow$ Exit loop.

6) Example: Printing No. from 1 to 5

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int i = 1;
```

```
    while (i <= 5)
```

```
    {
```

```
        printf("%d", i);
```

```
        i++;
```

```
    }
```

```
    getch();
```


3) Do-while loop Statement

1) The do-while loop is an exit controlled loop in which the condition is tested after executing the loop body

2) General Form (Syntax)

do	
{	
Statement(s);	
} while (condition);	

3) Even condition is false at the beginning than also body of do-while loop is executed at least once

4) Working of do-while loop

1. The loop body is executed first
2. After executing the statements, the condition is evaluated.
3. If the condition is true, the loop repeats.
4. If the condition is false, the loop terminates.
5. Hence, the loop body is executed at least once even if the condition is false initially.

5) Flow of Execution

- Execute loop body
- Evaluate condition

- True $\rightarrow$ Repeat loop

- False $\rightarrow$ Exit loop

Example:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int i = 1;
```

```
    do
```

```
    {
```

```
        printf("%d", i);
```

```
        i++;
```

```
    }
```

```
    while (i <= 5);
```

* Difference between while-loop & do-while loop

While - loop

do-while loop

1) It is beginning condition. check statement

It is ending condition. check statement

2) Syntax:

```
while (test-condition)
```

```
{
```

```
    body of while-loop
```

```
}
```

Syntax:

```
do
```

```
{
```

```
    body of do-while loop
```

```
} while (test-condition);
```


- | | |
|---|---|
| 3) It is entry control loop | It is exit control loop |
| 4) body of while-loop will not executed at all when condition is false at the beginning | body of do while loop is executed least once if condition is false at the beginning |
| 5) Semicolon after condition is not required. | Semicolon after while(condition) mandatory |

2.3 Branching Statements.

Branching statements are used to transfer control from one part to the program to another, thereby changing the normal sequential flow of execution.

Types of Branching Statement

1. return
2. goto
3. break
4. continue

1) return Statement

A return statement is used to return from a function. A function can use this statement as a mechanism to return a value to its calling function.

general form Syntax: -

return expression;

Where expression is any value expression.

Example

return x;

2) goto Statement

1) goto statement is used to transfer control from one point to another point without condition.

2) goto statement requires a label to transfer control to another place.

3) General form

goto label,

⋮

label:

Statement;

Example:

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i = 1;
    Start:
    printf("%d ", i);
    i++;

    if (i != 5)
        goto Start;
}
```

3) Break Statement

The break statement is used to terminate a loop or switch statement immediately and transfer control to the statement.

- It can be used within for, while, do-while, or switch statement.
- The break statement is written simply as break.
- In case of inner loop, it terminates the control of inner loop only.

Syntax:

```
break;
```


Example:-

```

for(i=1; i<=10; i++)
{
    if (printf("%d", i)
        if (i==5)
        {
            break;
        }
    getch();
}

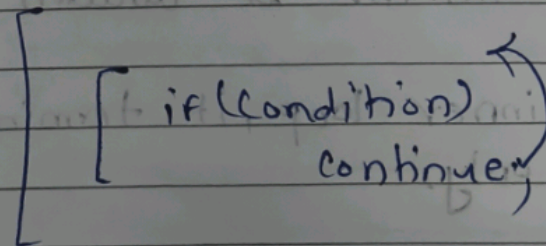
```

Continue Statement

Continue Statement is used in the loop to continue the loop with next iteration.

When continue statement is encounter in the loop, its transfer control to the beginning of the loop but with next iteration.

Continue Statement is used to continue inner loop as follows



Example 1

```
for (int i = 1; i <= 10; i++)  
{  
    if (i == 7)  
        continue;  
    else  
        printf("%d", i);  
}  
return 0;  
}
```